ASSIGNMENT – 2

PROGRAMMING IN C AND DATA STRUCTURE

(MCA-101)

UNIT II PROJECT

SIMPLE BANKING SYSTEM – VERSION 2

A menu-driven C application using arrays, strings, pointers,
structures and file handling

Submitted To: _____

Submitted By: _____

Course / Semester: MCA – Semester I

Roll Number: _____

College: _____

Academic Session: 2026–2027

INDEX

S. No.	Topic
1	Introduction and project overview
2	Objectives and scope
3	Problem statement and proposed solution
4	Unit II concepts implemented
5	System requirements and design
6	Data representation and program modules
7	Algorithm and program workflow
8	Complete C source code – Version 2
9	Explanation of important code sections
10	Sample output and test cases
11	Advantages, limitations and future scope
12	Conclusion and learning outcomes

1. INTRODUCTION AND PROJECT OVERVIEW

The Simple Banking System – Version 2 is an improved console-based application developed in the C programming language. The first version of the project provided basic banking operations such as login, balance enquiry, deposit, withdrawal, fund transfer and account information. It mainly demonstrated the use of variables, conditional statements, loops and switch-case selection.

Version 2 improves the internal design of the same project and connects it with selected concepts from Unit II of the MCA-101 syllabus. The purpose is not to implement every topic in Unit II, but to use the concepts that naturally fit a small banking application. The updated program uses an array of structures to store multiple account records, character arrays to store account-holder names, pointers to pass account data to functions, and file handling to preserve account information between program runs.

The application starts by loading previously saved account records from a binary file. If the file does not exist, the program uses a small set of sample accounts. A user must enter a valid account number and PIN. After successful login, a menu is displayed. The user can check the balance, deposit money, withdraw money, transfer money, view account information, view the demonstration account list, or save and exit.

This is an educational demonstration, not a real banking product. The sample PINs are stored in the program data and the file is not encrypted. A production banking application would require secure password hashing, authorization controls, transaction logging, database storage, audit trails and network security.

1.1 Project Overview

Project detail	Description
Project name	Simple Banking System – Version 2
Language	C
Application type	Menu-driven console application
Main Unit II concepts	Arrays, strings, pointers, structures and file handling
Data storage	Array in memory and binary file
Main users	Sample account holders
Execution	Compile and run through a C compiler / terminal

1.2 Difference Between Version 1 and Version 2

Feature	Version 1	Version 2
Account records	One account represented by separate variables	Multiple accounts represented using an array of structures
Account holder name	Not maintained	Stored as a character array inside structure
Functions	Most logic placed inside main()	Separate functions for login and each operation
Pointers	Not used	Pointers pass account data to functions
Transfer	Validates a receiver number but does not update another account	Finds receiver and updates both balances
Persistence	Data is lost when program ends	Account records can be saved and loaded from a file
Searching	No account collection to search	Linear search through the account array

2. PROJECT OBJECTIVES AND SCOPE

2.1 Objectives

- To upgrade the existing banking program without making its operation unnecessarily complicated.

- To demonstrate how arrays can hold multiple records of the same type.
- To use structures to group account number, account-holder name, PIN and balance.
- To process account-holder names using character arrays and string formatting.
- To use pointers when passing account records to functions.
- To divide the program into small functions so that each function performs one clear task.
- To implement a simple account search using a linear-search loop.
- To use file handling functions to save and reload account data.
- To validate amounts and account numbers before changing balances.
- To provide sample test cases for normal and invalid transactions.

2.2 Scope

The program supports a limited number of sample accounts. Each account has a unique account number, a name, a PIN and a balance. Only one user is logged in at a time. The logged-in user can perform transactions on their own account. The transfer option searches for a receiver account in the same in-memory array and updates the sender and receiver balances.

The file stores the account count and the account records in binary format. This makes the project useful for demonstrating fopen(), fread(), fwrite() and fclose(). The program does not use a database management system. It is deliberately kept as a small C-language assignment so that the relationship between the syllabus concepts and the program remains clear.

2.3 Limitations of Scope

- The program has a fixed maximum capacity of five accounts.
- Sample account records and PINs are provided for classroom demonstration.
- The application does not provide account creation or PIN-change features.
- The binary file is not encrypted and must not be used for real financial information.
- The program does not maintain a detailed transaction-history file.

3. PROBLEM STATEMENT AND PROPOSED SOLUTION

3.1 Problem Statement

The original Version 1 program demonstrates basic banking operations, but it stores only one account. Its account number and balance are maintained as separate variables. This approach becomes difficult to extend when multiple customers need to be represented. The transfer operation also checks a receiver account number but cannot credit another account because no collection of receiver accounts is maintained. In addition, all information is lost when the program terminates.

The updated problem is to design a simple banking application that can manage a small collection of accounts, identify an account by its number, perform basic transactions, and preserve the account data for the next execution. The solution should use selected Unit II topics in a visible and understandable manner.

3.2 Proposed Solution

The solution defines an Account structure and creates an array of Account structures. Each array element represents one customer account. A function named findAccount() searches the array and returns the index of the matching account. The login() function validates the account number and PIN. Transaction functions receive a pointer to the selected account, allowing them to read or update the original structure without copying the complete record.

The transfer() function uses the account number entered by the user to find the receiver. If the receiver exists, the function checks the amount and available balance. It then deducts the amount from the sender and credits the receiver. Finally, saveAccounts() writes the account count and records to a binary file, while loadAccounts() reads them when the program starts.

4. UNIT II CONCEPTS IMPLEMENTED IN THE PROJECT

4.1 Arrays

An array is a collection of elements of the same data type stored in consecutive memory locations. In this project, the declaration Account accounts[MAX_ACCOUNTS] creates an array capable of storing five Account structure variables. The first three positions contain sample accounts. The variable accountCount stores how many account records are currently in use.

```
Account accounts[MAX_ACCOUNTS];
int accountCount = 3;
```

The array is processed using loops. The findAccount() function checks each account number until it finds a match or reaches the end of the active records. The showAllAccounts() function also uses a for loop to display all available demonstration accounts.

4.2 Strings

A string in C is a sequence of characters terminated by the null character '\0'. C does not provide a separate built-in string data type. Character arrays are used to store strings. The Account structure contains char name[NAME_LENGTH], which stores the account-holder name. The sample values such as Aarav Sharma and Priya Verma are string literals copied into these character arrays during initialization.

```
char name[NAME_LENGTH];
```

The program uses the %s format specifier with printf() to display a name. The string.h header is included for standard string operations if the project is extended. The current version does not need to call strcpy() or strlen() because names are initialized directly in the sample records.

4.3 Structures

A structure is a user-defined data type that groups variables of different data types under one name. The Account structure combines an integer account number, a character array for the customer name, an integer PIN and a double balance. This is more organized than maintaining separate variables for every field of every account.

```
typedef struct {
    int accountNo;
    char name[NAME_LENGTH];
    int pin;
    double balance;
} Account;
```

The typedef keyword allows the program to use Account as a type name without writing struct Account every time. Structure members are accessed using the dot operator when the variable itself is used, for example accounts[i].accountNo.

4.4 Pointers

A pointer is a variable that stores the memory address of another variable. In this program, pointers are used to pass account structures to functions. The function checkBalance() receives const Account *account. The const keyword indicates that this function should not modify the account through this pointer. Functions such as deposit() and withdraw() receive Account *account because they need to update the balance.

```
void deposit(Account *account) {
```

```
    account->balance += amount;
}
```

The arrow operator (->) accesses a structure member through a pointer. The expression account->balance is equivalent to (*account).balance. In login(), an int pointer is used as an output parameter so the function can return the selected account index through the argument.

4.5 File Handling

File handling allows a program to store information permanently in a file. The program includes stdio.h and uses a FILE pointer to work with the data file named bank_accounts.dat. The saveAccounts() function opens the file in binary write mode using fopen(FILE_NAME, "wb"). It writes the account count and then the active account structures using fwrite().

The loadAccounts() function opens the same file in binary read mode using fopen(FILE_NAME, "rb"). It reads the stored count and account records using fread(). If the file is not present, the function returns and the sample accounts remain available. After file operations, fclose() closes the file and releases the associated resource.

Function	Purpose in this project
fopen()	Opens the binary data file for reading or writing.
fread()	Reads account count and account records from the file.
fwrite()	Writes account count and account records to the file.
fclose()	Closes the opened file.
FILE *	File pointer used to access the file.

4.6 Functions and Modular Programming

The program is divided into functions such as login(), deposit(), withdraw(), transfer(), showAccount(), findAccount(), saveAccounts() and loadAccounts(). Function prototypes are written before main() so the compiler knows each function's return type and parameters. This makes the program easier to read, test and maintain.

The project does not implement every Unit II topic. Dynamic memory allocation functions such as malloc(), calloc() and free(), arrays of pointers, function pointers and functions returning pointers are not required for this fixed-size demonstration. Multi-dimensional arrays are also not necessary for the account data. These topics can be practised separately.

5. SYSTEM REQUIREMENTS AND DESIGN

5.1 Hardware Requirements

- Desktop computer or laptop.
- At least 2 GB RAM (the program itself requires very little memory).
- Keyboard and display.

5.2 Software Requirements

- Operating system: Windows, Linux or macOS.
- C compiler: GCC, Clang or a compatible C compiler.
- Text editor or IDE such as VS Code, Code::Blocks or Dev-C++.
- Terminal or command prompt for compiling and running the program.

5.3 Program Architecture

The application follows a simple modular structure. The main() function controls the overall sequence. It loads account data, calls the login function, displays the menu and routes the selected operation to the appropriate

function. The account array acts as the program's in-memory data store. The file-handling functions provide basic persistence.

Module	Responsibility
main()	Controls startup, login, menu and exit.
login()	Checks account number and PIN with a maximum of three attempts.
findAccount()	Searches the account array and returns a matching index.
deposit() / withdraw()	Validates and updates the logged-in account balance.
transfer()	Finds a receiver and updates both account balances.
showAccount() / showAllAccounts()	Displays account information.
saveAccounts() / loadAccounts()	Writes and reads account data from a binary file.

5.4 Data Flow

The user enters login details. The login function calls findAccount() to locate the account and compares the entered PIN with the stored PIN. After authentication, the selected array index is returned to main(). The menu operation then receives the address of the selected Account structure. If the user performs a transaction, the balance in the array is updated. When the user chooses Save and Exit, the program writes the current account array to the data file.

5.5 Sample Account Data

Account number	Account holder	Demo PIN	Opening balance
1001	Aarav Sharma	1234	Rs. 5,000.00
1002	Priya Verma	2345	Rs. 7,500.00
1003	Rahul Patel	3456	Rs. 6,200.00

These are dummy records created only for demonstration. They do not represent real customers or real bank accounts.

6. DATA REPRESENTATION AND PROGRAM MODULES

6.1 Account Structure Fields

Field	Data type	Description
accountNo	int	Unique sample account number.
name	char[50]	Account-holder name as a null-terminated string.
pin	int	Demo PIN used for classroom login.
balance	double	Current account balance.

6.2 Array Index and Account Lookup

The array index is used internally to access an account record. The user does not need to know the index; the user enters an account number. findAccount() compares the entered number with accountNo in each active array element. If a match is found, the function returns its index. If no match exists, it returns -1. Returning -1 is a common way to indicate that a search did not find a valid position.

6.3 Pointer Parameters

A pointer parameter allows a function to work directly with the original account structure. For example, deposit(&accounts[loggedInIndex]) passes the address of the logged-in account. Inside deposit(), account-

>balance refers to the balance field of that original record. Therefore, the change remains available after the function returns.

6.4 File Format

The data file is a binary file. First, one integer is written to represent accountCount. After that, accountCount number of Account records are written. The same order is used when reading the file. This simple format is suitable for a classroom demonstration, but binary structure files may not be portable across different compilers or computer architectures because structure padding and data representation can vary.

7. ALGORITHM AND PROGRAM WORKFLOW

7.1 Main Algorithm

1. Start the program.
2. Load account data from bank_accounts.dat if the file exists.
3. Display the application title.
4. Ask the user for account number and PIN.
5. Search for the account and validate the PIN. Allow at most three attempts.
6. If authentication fails, display access denied and terminate.
7. If authentication succeeds, display the banking menu.
8. Read the user's menu choice and validate that it is numeric.
9. Call the function corresponding to the selected operation.
10. Repeat the menu until the user selects Save and Exit.
11. Save account data to the binary file and terminate.

7.2 Deposit Algorithm

12. Read the deposit amount.
13. Check that the input is numeric.
14. If the amount is less than or equal to zero, reject it.
15. Otherwise add the amount to the account balance.
16. Display the updated balance.

7.3 Withdrawal Algorithm

17. Read the withdrawal amount.
18. Check that the input is numeric and greater than zero.
19. Compare the amount with the available balance.
20. If the amount exceeds the balance, reject the transaction.
21. Otherwise subtract the amount and display the new balance.

7.4 Fund Transfer Algorithm

22. Read the receiver's account number.
23. Search for the receiver using findAccount().
24. Reject the transfer if the receiver does not exist or is the sender's own account.
25. Read and validate the transfer amount.
26. Check that the sender has sufficient balance.
27. Deduct the amount from the sender and add it to the receiver.
28. Display the transfer confirmation and updated sender balance.

7.5 File Saving Algorithm

29. Open bank_accounts.dat in binary write mode.
30. If the file cannot be opened, display an error and return.
31. Write the number of active accounts.
32. Write all active Account records.
33. Close the file.

8. COMPLETE C SOURCE CODE – VERSION 2

The following is the complete source code for the updated project. It is designed as a terminal-based program and does not require a graphical frontend or database.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_ACCOUNTS 5
#define NAME_LENGTH 50
#define FILE_NAME "bank_accounts.dat"

typedef struct {
    int accountNo;
    char name[NAME_LENGTH];
    int pin;
    double balance;
} Account;

Account accounts[MAX_ACCOUNTS] = {
    {1001, "Aarav Sharma", 1234, 5000.00},
    {1002, "Priya Verma", 2345, 7500.00},
    {1003, "Rahul Patel", 3456, 6200.00}
};
int accountCount = 3;

/* Function prototypes */
int login(int *loggedInIndex);
void showMenu(void);
void checkBalance(const Account *account);
void deposit(Account *account);
void withdraw(Account *account);
void transfer(Account *sender);
void showAccount(const Account *account);
void showAllAccounts(void);
void saveAccounts(void);
void loadAccounts(void);
int findAccount(int accountNo);
void printLine(void);

int main(void) {
    int loggedInIndex = -1;
    int choice;

    loadAccounts();

    printf("\n====================================\n");
    printf("       SIMPLE BANKING SYSTEM V2\n");
    printf("====================================\n");

    if (!login(&loggedInIndex)) {
        printf("\nAccess denied. Too many incorrect attempts.\n");
        return 0;
    }

    do {
        showMenu();
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Please enter a number.\n");
            int ch;
```

```
            while ((ch = getchar()) != '\n' && ch != EOF) { }
            choice = 0;
            continue;
        }

        switch (choice) {
            case 1:
                checkBalance(&accounts[loggedInIndex]);
                break;
            case 2:
                deposit(&accounts[loggedInIndex]);
                break;
            case 3:
                withdraw(&accounts[loggedInIndex]);
                break;
            case 4:
                transfer(&accounts[loggedInIndex]);
                break;
            case 5:
                showAccount(&accounts[loggedInIndex]);
                break;
            case 6:
                showAllAccounts();
                break;
            case 7:
                saveAccounts();
                printf("Data saved. Thank you for using the system.\n");
                break;
            default:
                printf("Invalid choice. Select 1 to 7.\n");
        }
    } while (choice != 7);

    return 0;
}

int login(int *loggedInIndex) {
    int enteredId, enteredPin, attempts;

    for (attempts = 1; attempts <= 3; attempts++) {
        printf("\nEnter account number: ");
        if (scanf("%d", &enteredId) != 1) {
            printf("Invalid account number format.\n");
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) { }
            continue;
        }

        printf("Enter PIN: ");
        if (scanf("%d", &enteredPin) != 1) {
            printf("Invalid PIN format.\n");
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) { }
            continue;
        }

        int index = findAccount(enteredId);
        if (index != -1 && accounts[index].pin == enteredPin) {
            *loggedInIndex = index;
            printf("\nLogin successful. Welcome, %s!\n", accounts[index].name);
            return 1;
        }
        printf("Incorrect account number or PIN. Attempt %d of 3.\n", attempts);
    }
    return 0;
}

void showMenu(void) {
    printLine();
    printf("\n1. Check Balance\n");
    printf("2. Deposit Money\n");
    printf("3. Withdraw Money\n");
    printf("4. Transfer Money\n");
    printf("5. View My Account\n");
    printf("6. View All Accounts (Demo)\n");
    printf("7. Save and Exit\n");
    printLine();
}
```

```
void checkBalance(const Account *account) {
    printf("\nCurrent balance: Rs. %.2f\n", account->balance);
}

void deposit(Account *account) {
    double amount;
    printf("Enter deposit amount: Rs. ");
    if (scanf("%lf", &amount) != 1) {
        printf("Invalid amount format.\n");
        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF) { }
        return;
    }
    if (amount <= 0) {
        printf("Amount must be greater than zero.\n");
        return;
    }
    account->balance += amount;
    printf("Deposit successful. New balance: Rs. %.2f\n", account->balance);
}

void withdraw(Account *account) {
    double amount;
    printf("Enter withdrawal amount: Rs. ");
    if (scanf("%lf", &amount) != 1) {
        printf("Invalid amount format.\n");
        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF) { }
        return;
    }
    if (amount <= 0) {
        printf("Amount must be greater than zero.\n");
    } else if (amount > account->balance) {
        printf("Insufficient balance.\n");
    } else {
        account->balance -= amount;
        printf("Withdrawal successful. New balance: Rs. %.2f\n", account->balance);
    }
}

void transfer(Account *sender) {
    int receiverNo, receiverIndex;
    double amount;

    printf("Enter receiver account number: ");
    if (scanf("%d", &receiverNo) != 1) {
        printf("Invalid account number.\n");
        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF) { }
        return;
    }

    receiverIndex = findAccount(receiverNo);
    if (receiverIndex == -1 || receiverNo == sender->accountNo) {
        printf("Receiver account not found or invalid.\n");
        return;
    }

    printf("Enter transfer amount: Rs. ");
    if (scanf("%lf", &amount) != 1) {
        printf("Invalid amount format.\n");
        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF) { }
        return;
    }

    if (amount <= 0) {
        printf("Amount must be greater than zero.\n");
    } else if (amount > sender->balance) {
        printf("Insufficient balance.\n");
    } else {
        sender->balance -= amount;
        accounts[receiverIndex].balance += amount;
        printf("Transfer of Rs. %.2f to account %d completed.\n",
               amount, receiverNo);
        printf("Updated balance: Rs. %.2f\n", sender->balance);
    }
}
```

```
void showAccount(const Account *account) {
    printf("\nACCOUNT DETAILS\n");
    printLine();
    printf("Account number : %d\n", account->accountNo);
    printf("Account holder : %s\n", account->name);
    printf("Account status : Active\n");
    printf("Balance        : Rs. %.2f\n", account->balance);
}

void showAllAccounts(void) {
    int i;
    printf("\nDEMO ACCOUNT LIST\n");
    printLine();
    for (i = 0; i < accountCount; i++) {
        printf("%d | %-20s | Rs. %.2f\n",
               accounts[i].accountNo, accounts[i].name,
               accounts[i].balance);
    }
}

int findAccount(int accountNo) {
    int i;
    for (i = 0; i < accountCount; i++) {
        if (accounts[i].accountNo == accountNo)
            return i;
    }
    return -1;
}

void saveAccounts(void) {
    FILE *fp = fopen(FILE_NAME, "wb");
    if (fp == NULL) {
        printf("Error: unable to open file for saving.\n");
        return;
    }
    fwrite(&accountCount, sizeof(int), 1, fp);
    fwrite(accounts, sizeof(Account), accountCount, fp);
    fclose(fp);
}

void loadAccounts(void) {
    FILE *fp = fopen(FILE_NAME, "rb");
    if (fp == NULL)
        return; /* First run: use the sample accounts above. */

    int storedCount;
    if (fread(&storedCount, sizeof(int), 1, fp) == 1 &&
        storedCount > 0 && storedCount <= MAX_ACCOUNTS) {
        size_t readCount = fread(accounts, sizeof(Account), storedCount, fp);
        if (readCount == (size_t)storedCount)
            accountCount = storedCount;
    }
    fclose(fp);
}

void printLine(void) {
    printf("-----------------------------------\n");
}
```

9. EXPLANATION OF IMPORTANT CODE SECTIONS

9.1 Header Files and Macro Definitions

stdio.h provides standard input/output and file-handling declarations. stdlib.h provides general utilities. string.h is included for standard string support and possible future string operations. The #define statements create symbolic constants for the maximum number of accounts, name length and data-file name. Using constants avoids repeating unexplained numeric values throughout the program.

9.2 Structure and Array Initialization

The Account structure groups related information into one record. The global array accounts stores up to five records, while accountCount indicates that three records are initially active. This arrangement makes it possible to search for and transfer money between different sample accounts.

9.3 Login and Pointer Output Parameter

login() receives int *loggedInIndex. When a valid account is found, the function stores the matching array index through *loggedInIndex. The function returns 1 for success and 0 for failure. The login process uses a for loop to limit attempts to three. The PIN is compared only after the account number has been found.

9.4 Input Validation

The program checks the return value of scanf(). A return value of 1 indicates that one item was successfully read for the numeric inputs used here. If the input format is invalid, the program displays an error and clears the remaining characters from the input line. Transaction amounts must also be greater than zero. Withdrawals and transfers additionally require sufficient balance.

9.5 Transfer Between Two Structure Elements

The transfer function receives a pointer to the sender account. It locates the receiver using the receiver's account number. After validation, it updates two separate structure elements: sender->balance is reduced and accounts[receiverIndex].balance is increased. This is a functional improvement over Version 1, where the receiver was only checked and the sender balance was reduced.

9.6 Saving and Loading Data

saveAccounts() writes the active account count followed by the account records. loadAccounts() reads the same information in the same sequence. The file is opened in binary mode because fwrite() and fread() operate on blocks of bytes. The program checks whether fopen() returns NULL and checks the stored account count before accepting the records.

9.7 Why Some Unit II Topics Are Not Used

The project is intentionally limited to concepts that have a clear purpose in a small account-management application. Dynamic memory allocation is unnecessary because the account capacity is fixed. A two-dimensional array is unnecessary because each customer is represented more naturally by a structure. A function pointer would add complexity without improving the required banking operations. These syllabus topics should be demonstrated in separate small programs rather than inserted artificially into this project.

10. SAMPLE OUTPUT AND TEST CASES

10.1 Successful Login

```
==================================
       SIMPLE BANKING SYSTEM V2
==================================

Enter account number: 1001
Enter PIN: 1234

Login successful. Welcome, Aarav Sharma!
----------------------------------
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
```

```
5. View My Account
6. View All Accounts (Demo)
7. Save and Exit
-----------------------------------
```

10.2 Balance Enquiry

```
Current balance: Rs. 5000.00
```

10.3 Deposit Example

```
Enter deposit amount: Rs. 1500
Deposit successful. New balance: Rs. 6500.00
```

10.4 Withdrawal Example

```
Enter withdrawal amount: Rs. 1000
Withdrawal successful. New balance: Rs. 5500.00
```

10.5 Transfer Example

```
Enter receiver account number: 1002
Enter transfer amount: Rs. 500
Transfer of Rs. 500.00 to account 1002 completed.
Updated balance: Rs. 5000.00
```

10.6 Test Cases

Test ID	Input / condition	Expected result
TC-01	Account 1001, PIN 1234	Login successful.
TC-02	Incorrect PIN three times	Access denied.
TC-03	Deposit Rs. 1,500	Balance increases by Rs. 1,500.
TC-04	Deposit Rs. 0	Transaction rejected.
TC-05	Withdraw Rs. 1,000 when balance is sufficient	Balance decreases by Rs. 1,000.
TC-06	Withdraw more than available balance	Insufficient-balance message.
TC-07	Transfer Rs. 500 from 1001 to 1002	Sender decreases and receiver increases by Rs. 500.
TC-08	Transfer to unknown account number	Transfer rejected.
TC-09	Transfer to own account number	Transfer rejected.
TC-10	Enter alphabetic text for numeric menu choice	Invalid-input message; program continues.
TC-11	Save and run program again	Saved account records are loaded if file data is valid.
TC-12	Data file does not exist on first run	Sample accounts remain available.

10.7 Example Balance Calculation

Assume account 1001 starts with Rs. 5,000.00. The user deposits Rs. 1,500.00, withdraws Rs. 1,000.00 and transfers Rs. 500.00 to account 1002.

Operation	Account 1001 balance
Opening balance	Rs. 5,000.00
After deposit of Rs. 1,500	Rs. 6,500.00
After withdrawal of Rs. 1,000	Rs. 5,500.00
After transfer of Rs. 500	Rs. 5,000.00

The receiver account 1002 increases by Rs. 500.00 when the transfer is successful. These values illustrate the expected arithmetic of the transaction.

11. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

11.1 Advantages

- Multiple sample accounts can be represented using one array of structures.
- Code is divided into functions, making it easier to understand and maintain.
- Pointers allow transaction functions to update the original account records.
- Fund transfer updates both the sender and receiver balances.
- Basic input and transaction validation reduces common invalid operations.
- File handling allows account data to be retained between executions.
- The project demonstrates several Unit II concepts in one understandable application.

11.2 Limitations

- The maximum number of accounts is fixed at five.
- The sample PINs are stored in plain form and are not secure.
- The binary data file is not encrypted or protected against tampering.
- There is no transaction history, receipt generation or administrator role.
- The application is single-user and does not handle simultaneous transactions.
- The program uses a simple file format intended for the same program environment.

11.3 Future Enhancements

- Add account creation and PIN-change options.
- Add a transaction-history structure and store deposits, withdrawals and transfers.
- Use dynamic memory allocation to support a variable number of accounts.
- Add stronger input handling using fgets() and strtol() for robust parsing.
- Store PINs securely using an appropriate password-hashing method rather than plain text.
- Add backup and recovery features for the data file.
- Use a database and transaction controls in a larger application.

12. CONCLUSION AND LEARNING OUTCOMES

The Simple Banking System – Version 2 is an improvement over the original Version 1 program. The earlier version demonstrated the basic control flow of a banking menu. The updated version retains the same familiar banking operations but changes the way account information is represented, accessed and stored.

The project demonstrates arrays through the collection of account records, strings through account-holder names, structures through the Account data type, pointers through function parameters, and file handling through binary read and write operations. It also demonstrates modular programming, searching, validation and the use of function prototypes.

The transfer operation is more complete because the receiver account is searched and credited after validation. The save and load functions make the application more useful as a demonstration of persistent storage. Although the project is not suitable for real banking, it provides a practical way to understand how several C programming concepts work together in a small application.

12.1 Learning Outcomes

- Understand how to define and initialize an array of structures.
- Access structure members using the dot and arrow operators.
- Pass pointers to functions and modify data through pointer parameters.

- Use character arrays to store and display strings.
- Search an array using a linear-search algorithm.
- Open, read, write and close a binary file using standard C functions.
- Organize a program using function prototypes and separate modules.
- Validate transaction data and test normal and invalid program conditions.

APPENDIX A – HOW TO COMPILE AND RUN

Save the source code in a file named banking_system_v2.c. If GCC is installed, open a terminal in the folder containing the file and run:

```
gcc banking_system_v2.c -o banking_system_v2
```

On Windows, run the generated executable from Command Prompt:

```
banking_system_v2.exe
```

On Linux or macOS, run:

```
./banking_system_v2
```

The program creates or reads bank_accounts.dat in the current working directory. Keep this file in the same folder when testing persistence. The first run uses the sample account records if the data file is not present.

APPENDIX B – DEMONSTRATION LOGIN DETAILS

Account number	PIN	Account holder
1001	1234	Aarav Sharma
1002	2345	Priya Verma
1003	3456	Rahul Patel

These credentials are dummy classroom values. Do not use personal or real banking credentials in this project.